Prometheus — 3–4 YOE Scenario-Based Q&A

1. **Prometheus target is showing DOWN. What will you check?**

Ans: First, I check the Prometheus Targets page and identify which target is showing DOWN. Then I check the scrape error to understand whether the issue is related to network connectivity, endpoint, port, or authentication.
I also verify the target configuration in prometheus.yml and check whether the application or exporter is running on the expected port. After fixing the issue, I verify that the target changes to UP.
Simple ga gurthupettuko: Target DOWN lo unte → first Targets page chudu → scrape error identify cheyyi → network, endpoint and port check cheyyi → Prometheus configuration verify cheyyi → application or exporter running unda chudu → issue fix chesi target UP ayyinda verify cheyyi.
Eng: If the target is DOWN → first check the Targets page → identify the scrape error → verify network, endpoint, and port → check Prometheus configuration → confirm the application or exporter is running → fix the issue and verify the target becomes UP.
Example:
http://<prometheus-server>:9090/targets → target DOWN lo unda chudu.
Prometheus Targets page lo **Last Error** → scrape error exact ga em undo chudu.
curl http://<target-ip>:<port>/metrics → target metrics endpoint access avutunda check cheyyi.
kubectl get pods → application or exporter Pod Running lo unda chudu.
kubectl get svc → correct Service and port available unnaya check cheyyi.
Issue fix chesi → Prometheus Targets page refresh cheyyi → target **UP** ayyinda verify cheyyi.

---

2. **Prometheus is not collecting metrics from an application. What will you check?**

Ans: First, I check whether the application exposes a metrics endpoint such as /metrics. Then I verify whether Prometheus is configured to scrape that endpoint.
I also check the target status, scrape path, port, and service discovery configuration. If the endpoint is available but Prometheus is still not collecting metrics, I check the Prometheus configuration and logs.
Simple ga gurthupettuko: Metrics collect avvakapothe → first /metrics endpoint unda chudu → endpoint access avutunda check cheyyi → Prometheus scrape configuration verify cheyyi → path and port check cheyyi → target status chudu → service discovery and Prometheus logs verify cheyyi.
Eng: If Prometheus is not collecting metrics → first check whether the /metrics endpoint exists → verify endpoint access → check scrape configuration → verify path and port → check target status → verify service discovery and Prometheus logs.
Example:
curl http://<app-ip>:<port>/metrics → application metrics expose chesthunda check cheyyi.
curl http://<app-ip>:<port>/metrics → metrics response vastunda verify cheyyi.
http://<prometheus-server>:9090/targets → application target UP or DOWN chudu.
cat prometheus.yml → scrape job, target, port and metrics path correct ga unnaya check cheyyi.
Issue fix chesi → Prometheus Targets page lo target **UP** ayyinda → Prometheus query lo metrics vastunnaya verify cheyyi.

---

3. **Prometheus target is UP, but metrics are missing. What will you check?**

Ans: First, I verify that the target is actually being scraped successfully and check the last scrape time and scrape duration. Then I search for the expected metric name in the Prometheus UI.
I also verify whether the application or exporter is exposing the metric with the expected name and labels. If the metric is not exposed, I check the exporter or application configuration.
Simple ga gurthupettuko: Target UP unna metrics kanipinchakapothe → first scrape successful aa chudu → expected metric name search cheyyi → application or exporter metric expose chesthunda check cheyyi → metric name and labels verify cheyyi → exporter configuration chudu → issue fix chesi metric malli verify cheyyi.
Eng: If the target is UP but metrics are missing → first verify the scrape is successful → search for the expected metric name → check whether the application or exporter exposes the metric → verify metric names and labels → check exporter configuration → fix the issue and verify the metric again.
Example:
Prometheus UI → **Status → Targets** → target UP and last scrape successful aa chudu.
Prometheus query box lo → up → target metric available unda check cheyyi.
Prometheus query box lo → <expected_metric_name> → required metric vastunda chudu.
curl http://<target-ip>:<port>/metrics → metric actually endpoint lo unda verify cheyyi.
Metric name or exporter configuration wrong ayithe → correct chesi → Prometheus query lo metric malli verify cheyyi.

---

4. **Prometheus scrape is failing. What will you check?**

Ans: First, I check the target's scrape error to identify the exact reason for the failure. Then I verify network connectivity between Prometheus and the target.
I also check the scrape endpoint, port, protocol, timeout, and authentication configuration. If the target is reachable but scraping still fails, I review the Prometheus configuration and logs.
Simple ga gurthupettuko: Scrape fail ayithe → first scrape error chudu → Prometheus nunchi target reach avutunda check cheyyi → endpoint and port verify cheyyi → protocol, timeout and authentication check cheyyi → Prometheus configuration and logs chudu → issue fix chesi scrape successful ayyinda verify cheyyi.
Eng: If the scrape fails → first check the scrape error → verify target connectivity from Prometheus → check endpoint and port → verify protocol, timeout, and authentication → check Prometheus configuration and logs → fix the issue and verify the scrape succeeds.
Example:
Prometheus **Targets** page → **Last Error** → exact scrape error chudu.
curl http://<target-ip>:<port>/metrics → metrics endpoint Prometheus server nunchi access avutunda test cheyyi.
cat prometheus.yml → target address, port and scrape path correct ga unnaya check cheyyi.
docker logs <prometheus-container> → Prometheus scrape-related errors emaina unnaya chudu.
Issue fix chesi → Targets page lo **Last Scrape** successful ga unda verify cheyyi.

5. **Prometheus exporter is not working. What will you check?**

Ans: First, I check whether the exporter process or container is running and review its logs for errors. Then I verify the exporter port and /metrics endpoint.
I also check whether Prometheus is configured with the correct exporter address and port. If the exporter depends on another service or credentials, I verify those dependencies as well.
Simple ga gurthupettuko: Exporter work avvakapothe → first exporter running unda chudu → logs check cheyyi → exporter port and /metrics endpoint verify cheyyi → Prometheus correct address and port use chesthunda check cheyyi → dependencies and credentials verify cheyyi → issue fix chesi metrics verify cheyyi.
Eng: If the exporter is not working → first check whether the exporter is running → check logs → verify exporter port and /metrics endpoint → confirm Prometheus uses the correct address and port → verify dependencies and credentials → fix the issue and verify metrics.
Example:
kubectl get pods → exporter Pod Running lo unda chudu.
kubectl logs <exporter-pod> → exporter error emaina unda check cheyyi.
curl http://<exporter-ip>:<port>/metrics → exporter metrics expose chesthunda verify cheyyi.
kubectl get svc → exporter Service correct port tho unda chudu.
Prometheus **Targets** page → exporter target UP aa check cheyyi.
Issue fix chesi → /metrics response and Prometheus target **UP** verify cheyyi.

---

6. **Prometheus query is not returning the expected data. What will you check?**

Ans: First, I verify the metric name used in the query and confirm that the metric actually exists in Prometheus. Then I check the labels attached to that metric.
I also verify the selected time range and query conditions because an incorrect label filter or time range can return no data. If required, I simplify the query and test each condition separately.
Simple ga gurthupettuko: Query data ivvakapothe → first metric name correct aa chudu → metric Prometheus lo unda check cheyyi → labels verify cheyyi → time range check cheyyi → filters correct aa chudu → query ni simple ga test chesi issue identify cheyyi.
Eng: If the query does not return expected data → first verify the metric name → confirm the metric exists → check labels → verify the time range → check filters → simplify the query and identify the issue.
Example:
Prometheus query box lo → up → basic metric data vastunda chudu.
<metric_name> → required metric exist avutunda check cheyyi.
<metric_name>{job="myapp"} → job label correct ga unda verify cheyyi.
Prometheus **Graph** time range → required time period select chesi data vastunda chudu.
Query ni simple chesi → one-by-one filters add cheyyi → wrong filter identify chesi correct cheyyi.

---

7. **Prometheus alert is not firing. What will you check?**

Ans: First, I check whether the alert rule is loaded correctly and whether the expression is returning the expected result. Then I check the alert state to see whether it is inactive, pending, or firing.
I also verify the for duration, rule evaluation interval, and Alertmanager configuration. If the alert expression is correct but notifications are not received, I check Alertmanager logs and routing configuration.
Simple ga gurthupettuko: Alert fire avvakapothe → first alert rule loaded aa chudu → expression result check cheyyi → alert state inactive or pending aa identify cheyyi → for duration verify cheyyi → Alertmanager configuration chudu → notification routing and logs check cheyyi → issue fix chesi alert verify cheyyi.
Eng: If the alert is not firing → first verify the alert rule is loaded → check the expression result → identify whether the alert is inactive or pending → verify the for duration → check Alertmanager configuration → verify notification routing and logs → fix the issue and verify the alert.
Example:
Prometheus → **Alerts** → required alert rule unda chudu.
Prometheus query box lo → alert expression run cheyyi → condition true avutunda check cheyyi.
Prometheus → **Status → Rules** → rule loaded and evaluating avutunda verify cheyyi.
Alert state **Pending** lo unte → for duration complete ayyinda chudu.
Alert firing ayina notification raakapothe → Alertmanager logs and routing configuration check cheyyi.
Issue fix chesi → alert **Firing** and notification received ayyinda verify cheyyi.

---

8. **Prometheus is using very high memory. What will you check?**

Ans: First, I check Prometheus memory usage and identify whether the usage increased because of the number of time series, queries, or retention data. Then I check whether there is a high-cardinality problem.
I also review PromQL queries, scrape configuration, and retention settings. After identifying the cause, I reduce unnecessary metrics or labels and adjust the Prometheus configuration appropriately.
Simple ga gurthupettuko: Prometheus memory ekkuva use chesthunte → first memory usage chudu → time series ekkuva unnaya check cheyyi → high cardinality unda identify cheyyi → heavy queries check cheyyi → scrape and retention configuration verify cheyyi → unnecessary metrics reduce chesi memory verify cheyyi.
Eng: If Prometheus is using high memory → first check memory usage → verify the number of time series → identify high cardinality → check heavy queries → verify scrape and retention settings → reduce unnecessary metrics and verify memory usage.
Example:
docker stats prometheus → Prometheus container memory usage chudu.
Prometheus UI → **Status → TSDB Status** → time series count ekkuva unda check cheyyi.
Prometheus query → count({__name__=~".+"}) → metrics count high ga unda chudu.
Prometheus configuration → unnecessary scrape targets and metrics unnaya verify cheyyi.
High-cardinality labels or unnecessary metrics remove chesi → Prometheus memory usage malli verify cheyyi.

9. **Prometheus has a high-cardinality problem. What will you do?**

Ans: First, I identify which metrics are creating a very large number of unique time series. Then I check whether labels contain highly variable values such as request IDs, user IDs, or dynamic URLs.
I remove or reduce unnecessary high-cardinality labels and keep only the labels required for monitoring and alerting. After the change, I monitor the number of active time series and Prometheus resource usage.
Simple ga gurthupettuko: High cardinality vaste → first ekkuva time series create chesthunna metric identify cheyyi → labels chudu → dynamic values unnaya check cheyyi → unnecessary labels remove cheyyi → required labels matrame keep cheyyi → time series and memory usage malli verify cheyyi.
Eng: If there is a high-cardinality problem → first identify the metric creating many time series → check its labels → find dynamic values → remove unnecessary labels → keep only required labels → verify time series count and memory usage again.
Example:
Prometheus → **Status → TSDB Status** → active time series count chudu.
Metric labels lo → user_id, request_id, dynamic URL lantivi unnaya check cheyyi.
count by (label_name) (<metric_name>) → label values ekkuva unnaya identify cheyyi.
Application metric configuration → unnecessary dynamic labels remove cheyyi.
Configuration change chesi → Prometheus reload/restart tarvatha → time series and memory usage verify cheyyi.

---

10. **Prometheus service is down. What will you check?**

Ans: First, I check whether the Prometheus process or container is running and review the logs for startup errors. Then I verify the Prometheus configuration file for syntax or configuration problems.
I also check disk space, memory, permissions, and whether the configured storage path is available. After fixing the issue, I restart Prometheus and verify that the UI and targets are working.
Simple ga gurthupettuko: Prometheus down ayithe → first process or container running unda chudu → logs check cheyyi → configuration error unda verify cheyyi → disk and memory check cheyyi → storage path and permissions chudu → issue fix chesi Prometheus restart cheyyi → UI and targets verify cheyyi.
Eng: If Prometheus is down → first check whether the process or container is running → check logs → verify configuration errors → check disk and memory → verify storage path and permissions → fix the issue and restart Prometheus → verify the UI and targets.
Example:
systemctl status prometheus → Prometheus service running lo unda chudu.
journalctl -u prometheus → startup error emaina unda check cheyyi.
promtool check config prometheus.yml → configuration syntax correct ga unda verify cheyyi.
df -h → disk space available unda chudu.
Issue fix chesi → systemctl restart prometheus → systemctl status prometheus → service running ayyinda verify cheyyi.

---

11. **Prometheus configuration change is not taking effect. What will you check?**

Ans: First, I verify that the configuration file was modified correctly and that there are no syntax errors. Then I check whether Prometheus was reloaded or restarted after the configuration change.
I also verify that the new target or rule appears in the Prometheus UI. If the change is still not reflected, I check Prometheus logs for reload or configuration errors.
Simple ga gurthupettuko: Config change reflect avvakapothe → first file change correct aa chudu → syntax error unda check cheyyi → Prometheus reload or restart ayyinda verify cheyyi → new target or rule UI lo unda chudu → logs check cheyyi → issue fix chesi configuration reflect ayyinda verify cheyyi.
Eng: If the configuration change is not taking effect → first verify the file change → check for syntax errors → confirm Prometheus was reloaded or restarted → check the new target or rule in the UI → review logs → fix the issue and verify the configuration.
Example:
promtool check config prometheus.yml → configuration syntax correct aa check cheyyi.
systemctl status prometheus → Prometheus running unda chudu.
systemctl restart prometheus → configuration apply cheyyadaniki restart cheyyi.
Prometheus → **Status → Targets** → new target kanipistunda check cheyyi.
Prometheus → **Status → Rules** → new rule loaded ayyinda verify cheyyi.

---

12. **Prometheus is not discovering Kubernetes services or pods. What will you check?**

Ans: First, I check the Kubernetes service discovery configuration in Prometheus. Then I verify whether Prometheus has the required Kubernetes API permissions.
I also check whether the Pod or Service has the expected labels and annotations used by the discovery configuration. Finally, I check Prometheus logs and targets to identify discovery or permission errors.
Simple ga gurthupettuko: Kubernetes targets discover avvakapothe → first service discovery configuration chudu → Kubernetes API permissions verify cheyyi → Pod and Service labels check cheyyi → required annotations unnaya chudu → Prometheus logs and targets verify cheyyi → issue fix chesi target discover ayyinda check cheyyi.
Eng: If Kubernetes targets are not discovered → first check service discovery configuration → verify Kubernetes API permissions → check Pod and Service labels → verify required annotations → check Prometheus logs and targets → fix the issue and verify target discovery.
Example:
cat prometheus.yml → Kubernetes service discovery configuration correct ga unda chudu.
kubectl get pods --show-labels → required labels unnaya check cheyyi.
kubectl get svc --show-labels → Service labels verify cheyyi.
kubectl get clusterrole prometheus -o yaml → required Kubernetes permissions unnaya chudu.
Prometheus → **Status → Service Discovery** → targets discover avutunnaya check cheyyi.
Issue fix chesi → **Targets** page lo Kubernetes targets UP ayyaya verify cheyyi.

13. **Prometheus is running out of disk space. What will you check?**

Ans: First, I check the disk usage of the filesystem where Prometheus stores its TSDB data. Then I review the retention period and the amount of data being stored.
I also check whether the number of metrics and time series has increased significantly. After identifying the cause, I adjust retention or storage capacity appropriately and verify that Prometheus has enough free space.
Simple ga gurthupettuko: Prometheus disk full avutunte → first disk usage chudu → TSDB data ekkuva unda check cheyyi → retention period verify cheyyi → time series count chudu → unnecessary metrics identify cheyyi → retention or storage adjust cheyyi → free disk space verify cheyyi.
Eng: If Prometheus is running out of disk space → first check disk usage → verify TSDB data size → check retention period → check time series count → identify unnecessary metrics → adjust retention or storage → verify free disk space.
Example:
df -h → Prometheus disk usage chudu.
du -sh /prometheus → Prometheus data directory size check cheyyi.
Prometheus configuration → retention settings verify cheyyi.
Prometheus → **Status → TSDB Status** → time series count chudu.
Retention or storage capacity correct chesi → df -h → free disk space malli verify cheyyi.

---

14. **Prometheus is scraping too many targets. What will you check?**

Ans: First, I check the Targets page and identify how many targets Prometheus is currently scraping. Then I review the scrape configuration and service discovery configuration.
I identify duplicate, unnecessary, or unwanted targets and remove them from the configuration. After making the change, I verify the target count and Prometheus resource usage.
Simple ga gurthupettuko: Targets ekkuva unte → first total targets count chudu → scrape configuration check cheyyi → service discovery verify cheyyi → duplicate and unnecessary targets identify cheyyi → unwanted targets remove cheyyi → target count and resource usage verify cheyyi.
Eng: If Prometheus is scraping too many targets → first check the total target count → review scrape configuration → verify service discovery → identify duplicate and unnecessary targets → remove unwanted targets → verify target count and resource usage.
Example:
Prometheus → **Status → Targets** → total targets count chudu.
cat prometheus.yml → scrape jobs and static targets verify cheyyi.
Prometheus → **Status → Service Discovery** → unwanted discovered targets unnaya check cheyyi.
Unnecessary target configuration remove chesi → Prometheus reload cheyyi.
Targets page lo → unwanted targets remove ayyaya and resource usage normal ga unda verify cheyyi.

---

15. **Prometheus query is taking too much time. What will you check?**

Ans: First, I identify the query that is taking a long time and check how much data it is processing. Then I review the number of series selected by the query and the time range being used.
I also check whether the query is unnecessarily broad or using expensive operations. I optimize the query by reducing the time range, filtering labels, or using recording rules where appropriate.
Simple ga gurthupettuko: Query slow ga unte → first slow query identify cheyyi → time range chudu → enni time series select chesthundo check cheyyi → broad filters unnaya verify cheyyi → query simplify cheyyi → required ayithe recording rule use cheyyi → query performance malli verify cheyyi.
Eng: If a query is slow → first identify the slow query → check the time range → verify how many time series it selects → check broad filters → simplify the query → use a recording rule when required → verify query performance again.
Example:
Prometheus query → <large_metric_query> → query execution time chudu.
Time range → 30 days badulu 1 hour select chesi response fast avutunda check cheyyi.
Query labels → {job="myapp"} laga specific filter add chesi series count reduce cheyyi.
Prometheus query → unnecessary functions or broad selectors remove cheyyi.
Query still slow ayithe → recording rule use chesi → optimized query response verify cheyyi.

---

16. **Prometheus Alertmanager is not sending notifications. What will you check?**

Ans: First, I confirm that Prometheus is actually firing the alert. Then I check whether Prometheus is correctly connected to Alertmanager.
I verify the Alertmanager receiver, routing rules, notification configuration, and credentials if required. I also check Alertmanager logs to identify delivery failures.
Simple ga gurthupettuko: Notification raakapothe → first alert actually firing aa chudu → Prometheus to Alertmanager connection verify cheyyi → receiver configuration check cheyyi → routing rules verify cheyyi → credentials check cheyyi → Alertmanager logs chudu → issue fix chesi notification verify cheyyi.
Eng: If notifications are not being sent → first confirm the alert is firing → verify the Prometheus to Alertmanager connection → check receiver configuration → verify routing rules → check credentials → review Alertmanager logs → fix the issue and verify the notification.
Example:
Prometheus → **Alerts** → alert state **Firing** lo unda chudu.
Prometheus configuration → Alertmanager address correct ga unda verify cheyyi.
curl http://<alertmanager>:9093/-/healthy → Alertmanager healthy ga unda check cheyyi.
Alertmanager UI → **Alerts** → alert receive ayyinda chudu.
docker logs <alertmanager-container> → notification delivery error emaina unda check cheyyi.
Issue fix chesi → test alert fire cheyyi → notification received ayyinda verify cheyyi.

17. **Prometheus is showing stale or old metric values. What will you check?**

Ans: First, I check the last successful scrape time for the target. Then I verify whether the target is currently UP and whether the application is exposing fresh metric values.
I also check scrape intervals and whether the metric itself is updated by the application or exporter. If the target is down or the exporter is returning old data, I fix that issue and verify the metric again.
Simple ga gurthupettuko: Old data kanipisthe → first last successful scrape time chudu → target UP aa check cheyyi → application fresh metrics istunda verify cheyyi → scrape interval check cheyyi → exporter old data istunda chudu → issue fix chesi fresh metric verify cheyyi.
Eng: If old metric values are shown → first check the last successful scrape time → verify the target is UP → confirm the application exposes fresh metrics → check the scrape interval → verify whether the exporter returns old data → fix the issue and verify fresh metrics.
Example:
Prometheus → **Status → Targets** → Last Scrape time chudu.
curl http://<target>:<port>/metrics → current metric value ento check cheyyi.
Prometheus configuration → scrape_interval verify cheyyi.
Exporter logs → metrics update avutunnaya check cheyyi.
Issue fix chesi → target successful scrape ayyaka → Prometheus query lo fresh value verify cheyyi.

---

18. **Prometheus target is UP but scrape latency is very high. What will you check?**

Ans: First, I check the scrape duration shown for the target. Then I verify whether the /metrics endpoint itself is taking too long to respond.
I also check the number of metrics being exposed, application or exporter performance, network latency, and scrape timeout settings. After identifying the bottleneck, I reduce unnecessary metrics or improve the endpoint performance.
Simple ga gurthupettuko: Target UP unna scrape slow ayithe → first scrape duration chudu → /metrics response time check cheyyi → metrics count ekkuva unda verify cheyyi → exporter or application performance chudu → network and timeout check cheyyi → bottleneck fix chesi scrape duration verify cheyyi.
Eng: If the target is UP but scraping is slow → first check scrape duration → verify /metrics response time → check the number of metrics → check exporter or application performance → verify network and timeout → fix the bottleneck and verify scrape duration.
Example:
Prometheus → **Status → Targets** → Scrape Duration value chudu.
time curl http://<target>:<port>/metrics → metrics endpoint response time check cheyyi.
curl http://<target>:<port>/metrics | wc -l → metrics output ekkuva unda approximate ga chudu.
Target application/exporter CPU and memory usage → performance issue unda check cheyyi.
Issue fix chesi → Targets page lo scrape duration normal ayyinda verify cheyyi.

---

19. **Prometheus rules are not loading. What will you check?**

Ans: First, I check whether the rule file is correctly referenced in the Prometheus configuration. Then I validate the rule syntax and check whether Prometheus can load the file.
I also check the Prometheus Rules page and logs for rule evaluation or parsing errors. After correcting the rule file, I reload Prometheus and verify that the rules are loaded successfully.
Simple ga gurthupettuko: Rules load avvakapothe → first rule file configuration lo reference unda chudu → rule syntax verify cheyyi → Prometheus Rules page check cheyyi → logs lo parsing error chudu → rule correct chesi Prometheus reload cheyyi → rules loaded ayyaya verify cheyyi.
Eng: If Prometheus rules are not loading → first check the rule file reference → validate rule syntax → check the Rules page → review parsing errors in logs → correct the rule and reload Prometheus → verify the rules are loaded.
Example:
cat prometheus.yml → rule_files configuration correct ga unda chudu.
promtool check rules rules.yml → rule syntax correct aa verify cheyyi.
Prometheus → **Status → Rules** → rule loaded ayyinda chudu.
journalctl -u prometheus → rule parsing error emaina unda check cheyyi.
Issue fix chesi → Prometheus reload/restart cheyyi → **Status → Rules** lo rule active ga unda verify cheyyi.

---

20. **How will you troubleshoot a Prometheus production monitoring issue?**

Ans: First, I check whether Prometheus itself is healthy and whether the required targets are UP. Then I identify whether the issue is related to scraping, missing metrics, queries, alerts, storage, or Alertmanager.
I review recent configuration changes and Prometheus logs, and then troubleshoot the affected component step by step. After fixing the root cause, I verify targets, metrics, queries, and alerts to make sure monitoring is working correctly again.
Simple ga gurthupettuko: Prometheus Prod issue vaste → first Prometheus healthy aa chudu → targets UP unnaya check cheyyi → scrape, metrics, query, alert or storage issue identify cheyyi → recent changes and logs check cheyyi → affected component troubleshoot cheyyi → root cause fix chesi targets, metrics and alerts verify cheyyi.
Eng: If there is a Prometheus production issue → first check Prometheus health → verify targets are UP → identify whether the issue is scraping, metrics, queries, alerts, or storage → review recent changes and logs → troubleshoot the affected component → fix the root cause and verify targets, metrics, and alerts.
Example:
systemctl status prometheus → Prometheus service healthy ga unda chudu.
Prometheus → **Status → Targets** → important targets UP unnaya check cheyyi.
Prometheus query → up → targets metrics available unnaya verify cheyyi.
Prometheus → **Alerts** → alerts firing or notification issue unda chudu.
df -h → Prometheus storage full ayyinda check cheyyi.
Issue identify chesi → required fix apply cheyyi → targets, metrics and alerts anni working ga unnaya final ga verify cheyyi.